DISTRIBUTED COMPUTING

Practical Programs & CSV Datasets — Topics 1 to 10

How to use: For each practical, create the CSV file using the exact filename shown, save the Python program in the same folder, and run it in the VS Code terminal. The programs use Python standard-library modules only.

Note: Extra CSV rows beyond the manual's sample are illustrative data to make the programs runnable. For MPI, the manual supplies only one matrix; this booklet demonstrates A × A-transpose as an explicit example choice.

Topic 1: RPC / RMI

Aim: CSV rows represent client requests. The dispatcher checks the method name, executes the matching function, and returns its result. This is a local simulation, not real network RPC/RMI. The processData function is illustrative.

CSV filename: rpc_calls_1.csv

CSV dataset

```
call_id,client_name,method_name,param_value,timestamp
101,Client_A,processData,450,10:00:01
102,Client_B,processData,200,10:00:02
103,Client_C,processData,300,10:00:03
```

Python program

```
import csv

def processData(value):
    return value * 2

with open("rpc_calls_1.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        call_id = row["call_id"]
        client = row["client_name"]
        method = row["method_name"]
        parameter = int(row["param_value"])

        print("\\n[DISPATCH]")
        print(client, "invoking", method)

        request = {
            "call_id": call_id,
            "method": method,
            "parameter": parameter
        }

        print("[SERVER EXECUTION]")
        if request["method"] == "processData":
            result = processData(request["parameter"])
        else:
            result = "Method not found"

        print("[RETURN]")
        print("Result:", result)
```

Key points: Memory: CSV → DISPATCH → EXECUTE → RETURN.

Topic 2: Multithreading with Mutex / Locks

Aim: Each CSV row becomes a thread task. sleep() simulates work. The lock protects shared status and completion-list updates. Thread output order may vary.

CSV filename: tasks_4.csv

CSV dataset

```
task_id,thread_name,burst_ms,priority,status
T1,Worker_1,200,HIGH,PENDING
T2,Worker_2,300,MEDIUM,PENDING
T3,Worker_3,150,LOW,PENDING
T4,Worker_4,250,HIGH,PENDING
```

Python program

```
import csv
import threading
import time

lock = threading.Lock()
completed_tasks = []

def process_task(task):
    task_id = task["task_id"]
    worker = task["thread_name"]
    burst = int(task["burst_ms"])

    print(f"\\n[{worker}] Started processing {task_id}")
    time.sleep(burst / 1000)

    lock.acquire()
    print(f"[{worker}] Lock acquired")
    task["status"] = "COMPLETED"
    completed_tasks.append(task_id)
    print(f"[{worker}] Completed {task_id}")
    lock.release()
    print(f"[{worker}] Lock released")

with open("tasks_4.csv", "r") as file:
    tasks = list(csv.DictReader(file))

threads = []
for task in tasks:
    t = threading.Thread(
        target=process_task,
        args=(task,),
        name=task["thread_name"]
    )
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("\\n===== FINAL COMPLETED TASKS =====")
for task in tasks:
    print(task["task_id"], ":", task["status"])
print("\\nAll tasks completed:", completed_tasks)
```

Key points: Memory: READ → THREAD → START → LOCK → UPDATE → UNLOCK → JOIN.

Topic 3: Lamport Logical Clock

Aim: Internal and send events increment the local clock. A receive event uses max(local clock, received timestamp) + 1. Equal timestamps do not establish causality; the sort tie-break is only for display.

CSV filename: events_7.csv

CSV dataset

```
event_id,process_id,event_type,message_to,initial_clock
E1,P1,INTERNAL,NONE,0
E2,P1,SEND,P2,0
E3,P2,RECEIVE,P1,0
E4,P2,INTERNAL,NONE,0
E5,P3,INTERNAL,NONE,0
```

Python program

```
import csv

clocks = {}
messages = {}
event_records = []

with open("events_7.csv", "r") as file:
    for row in csv.DictReader(file):
        event_id = row["event_id"]
        process = row["process_id"]
        event_type = row["event_type"].upper()
        message_to = row["message_to"].strip()
        initial = int(row["initial_clock"])

        if process not in clocks:
            clocks[process] = initial

        if event_type == "INTERNAL":
            clocks[process] += 1
        elif event_type == "SEND":
            clocks[process] += 1
            messages[(process, message_to)] = clocks[process]
            print("\\n", process, "sends to", message_to,
                  "timestamp:", clocks[process])
        elif event_type == "RECEIVE":
            key = (message_to, process)
            if key in messages:
                msg_time = messages.pop(key)
                clocks[process] = max(clocks[process], msg_time) + 1
            else:
                clocks[process] += 1
                print("Warning: message timestamp not found")

        print(event_id, process, event_type, "Clock =", clocks[process])
        event_records.append((event_id, process, clocks[process]))

event_records.sort(key=lambda x: (x[2], x[1], x[0]))
print("\\n===== FINAL ORDER =====")
for event in event_records:
    print(event[0], "->", event[1], "->", event[2])
```

Key points: Memory: INTERNAL=+1, SEND=+1, RECEIVE=MAX+1.

Topic 4: Cristian's Clock Synchronization

Aim: RTT = T1 – T0. Estimated server time = T_server + RTT/2. Offset = estimated server time – T1. Assumes roughly symmetric network delay.

CSV filename: cristian_clocks_8.csv

CSV dataset

```
client_id,t0_send_ms,t_server_ms,t1_recv_ms
C1,1000,1050,1100
C2,2000,2070,2120
C3,3000,3050,3080
```

Python program

```
import csv

with open("cristian_clocks_8.csv", "r") as file:
    for row in csv.DictReader(file):
        client = row["client_id"]
        t0 = int(row["t0_send_ms"])
        server = int(row["t_server_ms"])
        t1 = int(row["t1_recv_ms"])

        rtt = t1 - t0
        estimated_time = server + (rtt / 2)
        offset = estimated_time - t1

        print("\\nClient:", client)
        print("RTT:", rtt, "ms")
        print("Estimated Server Time:", estimated_time, "ms")
        print("Clock Offset:", offset, "ms")

        if offset > 0:
            print("Client clock is behind by", offset, "ms")
        elif offset < 0:
            print("Client clock is ahead by", abs(offset), "ms")
        else:
            print("Client clock is synchronized")
        print("----------------------------")
```

Key points: Memory: READ → RTT → HALF RTT → ESTIMATE → OFFSET.

Topic 5: Berkeley Clock Synchronization

Aim: The program discards faulty nodes, averages valid clocks, and calculates each offset. Sample average = 3605 seconds; offsets are N1 +5, N2 –5, N3 +15, N4 –15. It reports adjustments but does not alter OS clocks.

CSV filename: node_clocks_3.csv

CSV dataset

```
node_id,clock_time_sec,is_faulty
N1,3600,FALSE
N2,3610,FALSE
N3,3590,FALSE
N4,3620,FALSE
N5,5000,TRUE
```

Python program

```
import csv

nodes = []

with open("node_clocks_3.csv", "r") as file:
    for row in csv.DictReader(file):
        node_id = row["node_id"]
        clock = int(row["clock_time_sec"])
        faulty = row["is_faulty"].upper() == "TRUE"
        print(node_id, "Clock:", clock)

        if faulty:
            print(node_id, "is faulty - discarded")
        else:
            nodes.append({"node_id": node_id, "clock": clock})

average = sum(n["clock"] for n in nodes) / len(nodes)
print("\\nAverage Clock Time:", average)
print("\\n===== CLOCK ADJUSTMENTS =====")

for node in nodes:
    adjustment = average - node["clock"]
    print(node["node_id"], "Adjustment:", adjustment, "seconds")
    if adjustment > 0:
        print("Clock must move forward")
    elif adjustment < 0:
        print("Clock must move backward")
    else:
        print("Clock is already synchronized")

print("\\nSynchronization Completed")
```

Key points: Memory: READ → FILTER → AVERAGE → OFFSET → ADJUST.

Topic 6: Vector Clocks

Aim: Each process maintains a vector. On receive, merge component-wise maxima, then increment the receiving process's component. A happens-before relation requires all components ≤ and at least one <; otherwise the pair is concurrent.

CSV filename: vector_events_6.csv

CSV dataset

```
event_id,process_id,event_action,target_process
E1,P1,INTERNAL,NONE
E2,P1,SEND,P2
E3,P2,RECEIVE,P1
E4,P2,INTERNAL,NONE
E5,P3,INTERNAL,NONE
E6,P3,SEND,P1
E7,P1,RECEIVE,P3
```

Python program

```
import csv

index = {"P1": 0, "P2": 1, "P3": 2}
clocks = {"P1": [0, 0, 0], "P2": [0, 0, 0], "P3": [0, 0, 0]}
messages = {}
event_vectors = []

with open("vector_events_6.csv", "r") as file:
    for row in csv.DictReader(file):
        event_id = row["event_id"]
        process = row["process_id"]
        action = row["event_action"].upper()
        target = row["target_process"].strip()
        i = index[process]

        if action == "INTERNAL":
            clocks[process][i] += 1
        elif action == "SEND":
            clocks[process][i] += 1
            messages[(process, target)] = clocks[process].copy()
        elif action == "RECEIVE":
            key = (target, process)
            if key in messages:
                received = messages.pop(key)
                for j in range(3):
                    clocks[process][j] = max(
                        clocks[process][j], received[j])
            clocks[process][i] += 1

        print(event_id, process, action, "Vector:", clocks[process])
        event_vectors.append((event_id, clocks[process].copy()))

def happens_before(a, b):
    return (all(x <= y for x, y in zip(a, b))
            and any(x < y for x, y in zip(a, b)))

print("\\n===== CAUSAL / CONCURRENT EVENTS =====")
for i in range(len(event_vectors)):
    for j in range(i + 1, len(event_vectors)):
        id1, v1 = event_vectors[i]
        id2, v2 = event_vectors[j]
        if happens_before(v1, v2):
            print(id1, "<", id2, "Causally Preceding")
        elif happens_before(v2, v1):
            print(id2, "<", id1, "Causally Preceding")
        else:
            print(id1, "||", id2, "Concurrent")
```

Key points: Memory: INTERNAL/SEND own component +1; RECEIVE = component-wise MAX, then own +1.

Topic 7: Ring Algorithm — Leader Election

Aim: The code follows next_node_id around the ring, skips inactive nodes, and elects the highest active node ID. The sample elects Node 4. It is a CSV simulation, not real network token passing.

CSV filename: ring_nodes_55.csv

CSV dataset

```
node_id,next_node_id,is_active,priority_rank
1,2,TRUE,85
2,3,TRUE,70
3,4,TRUE,90
4,5,TRUE,60
5,1,FALSE,95
```

Python program

```
import csv

nodes = {}
with open("ring_nodes_55.csv", "r") as file:
    for row in csv.DictReader(file):
        node_id = int(row["node_id"])
        nodes[node_id] = {
            "next": int(row["next_node_id"]),
            "active": row["is_active"].upper() == "TRUE"
        }

coordinator_id = 5
print("Coordinator Node", coordinator_id, "has failed!")
print("Starting Ring Election...\\n")

current = 1
visited = set()
active_nodes = []

print("Election token is circulating:")
while current not in visited:
    visited.add(current)
    if nodes[current]["active"]:
        print("Token reached Node", current)
        active_nodes.append(current)
    else:
        print("Node", current, "is inactive. Skipping...")
    current = nodes[current]["next"]

if active_nodes:
    new_coordinator = max(active_nodes)
    print("\\nActive Nodes:", active_nodes)
    print("New Coordinator:", new_coordinator)
    print("\\nCOORDINATOR message broadcast:")
    for node in active_nodes:
        print("Node", node, "recognizes Node",
              new_coordinator, "as Coordinator")
else:
    print("No active nodes available!")
```

Key points: Memory: FAILURE → TOKEN → CIRCULATION → HIGHEST ID → COORDINATOR.

Topic 8: Bully Algorithm — Leader Election

Aim: The initiator sends ELECTION messages to higher-ID nodes. Active higher nodes reply OK. The highest active node broadcasts COORDINATOR. The sample elects Node 4. This is a simplified CSV simulation.

CSV filename: bully_cluster_125.csv

CSV dataset

```
node_id,is_alive,is_current_coordinator
1,TRUE,FALSE
2,TRUE,FALSE
3,TRUE,FALSE
4,TRUE,FALSE
5,FALSE,TRUE
```

Python program

```
import csv

nodes = {}
with open("bully_cluster_125.csv", "r") as file:
    for row in csv.DictReader(file):
        node_id = int(row["node_id"])
        nodes[node_id] = {
            "alive": row["is_alive"].upper() == "TRUE",
            "coordinator": row["is_current_coordinator"].upper() == "TRUE"
        }

coordinator = None
for node_id in nodes:
    if nodes[node_id]["coordinator"]:
        coordinator = node_id
        break

print("Current Coordinator:", coordinator)
if coordinator is not None and not nodes[coordinator]["alive"]:
    print("Coordinator", coordinator, "has failed!")

initiator = 2
if initiator not in nodes or not nodes[initiator]["alive"]:
    print("Initiator is not active!")
else:
    print("\\nNode", initiator, "starts the election")
    active_nodes = [n for n in nodes if nodes[n]["alive"]]
    participants = [n for n in active_nodes if n >= initiator]

    for node in participants:
        print("\\nNode", node, "sends ELECTION messages")
        higher_nodes = [n for n in nodes if n > node]
        received_ok = False
        for higher in higher_nodes:
            print("ELECTION sent to Node", higher)
            if nodes[higher]["alive"]:
                print("Node", higher, "replies: OK")
                received_ok = True
            else:
                print("Node", higher, "is not responding")
        if not received_ok:
            print("Node", node, "received no OK response")

    new_coordinator = max(active_nodes)
    print("\\n===== NEW COORDINATOR =====")
    print("Node", new_coordinator, "is elected as Coordinator")
    print("\\nCOORDINATOR message broadcast:")
    for node in active_nodes:
        print("Node", node, "recognizes Node",
              new_coordinator, "as Coordinator")
```

Key points: Memory: FAILURE → ELECTION → OK → HIGHEST ID → COORDINATOR.

Topic 9: Dynamic Load Balancing — Round Robin

Aim: i % len(workers) cycles through workers. For the sample, Worker_1 gets 65 units, Worker_2 gets 45, and Worker_3 gets 95. Round Robin balances job count, not necessarily compute cost. Assignments are not written back to the CSV.

CSV filename: workload_13.csv

CSV dataset

```
job_id,compute_units,assigned_worker,status
J101,35,UNASSIGNED,PENDING
J102,20,UNASSIGNED,PENDING
J103,45,UNASSIGNED,PENDING
J104,30,UNASSIGNED,PENDING
J105,25,UNASSIGNED,PENDING
J106,50,UNASSIGNED,PENDING
```

Python program

```
import csv

workers = ["Worker_1", "Worker_2", "Worker_3"]
worker_load = {"Worker_1": 0, "Worker_2": 0, "Worker_3": 0}
job_count = {"Worker_1": 0, "Worker_2": 0, "Worker_3": 0}

with open("workload_13.csv", "r") as file:
    jobs = list(csv.DictReader(file))

print("===== INCOMING JOB ALLOCATION =====")
for i, job in enumerate(jobs):
    worker = workers[i % len(workers)]
    job["assigned_worker"] = worker
    job["status"] = "ASSIGNED"
    compute = int(job["compute_units"])
    worker_load[worker] += compute
    job_count[worker] += 1
    print(job["job_id"], "->", worker, "| Compute Units:", compute)

print("\\n===== CURRENT WORKER LOAD =====")
for worker in workers:
    print(worker, "| Jobs:", job_count[worker],
          "| Total Load:", worker_load[worker])

print("\\n===== LOAD BALANCING SUMMARY =====")
for worker in workers:
    print(worker, "has", job_count[worker], "jobs")
print("\\nAll jobs allocated successfully!")
```

Key points: Memory: READ → SELECT WORKER → ASSIGN → UPDATE LOAD → SUMMARY.

Topic 10: MPI Matrix Multiplication — Simulated Scatter-Gather

Aim: The master distributes one row per worker task (Scatter). Each worker calculates dot products. The master collects the returned rows (Gather). Expected product: [46, 36, 23, 31]; [36, 30, 21, 24]; [23, 21, 30, 28]; [31, 24, 28, 39].

CSV filename: matrix_38.csv

CSV dataset

```
row_id,col_0,col_1,col_2,col_3
R0,2,4,1,5
R1,1,3,2,4
R2,3,2,4,1
R3,5,1,2,3
```

Python program

```
import csv
from concurrent.futures import ThreadPoolExecutor

def calculate_row(task):
    row_id, row, columns = task
    print("Worker processing row:", row_id)
    result = []
    for column in columns:
        value = sum(a * b for a, b in zip(row, column))
        result.append(value)
    print("Worker completed row:", row_id)
    return row_id, result

matrix = []
row_ids = []

with open("matrix_38.csv", "r") as file:
    for row in csv.DictReader(file):
        row_ids.append(row["row_id"])
        values = [
            int(row["col_0"]), int(row["col_1"]),
            int(row["col_2"]), int(row["col_3"])
        ]
        matrix.append(values)

# This example calculates A multiplied by A-transpose.
columns = list(zip(*matrix))

print("===== MASTER: SCATTERING MATRIX ROWS =====")
tasks = []
for i in range(len(matrix)):
    print("Sending row", row_ids[i], "to worker")
    tasks.append((row_ids[i], matrix[i], columns))

print("\\n===== WORKERS: COMPUTING =====")
with ThreadPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(calculate_row, tasks))

print("\\n===== MASTER: GATHERING RESULTS =====")
results.sort(key=lambda x: row_ids.index(x[0]))
final_matrix = []

for row_id, result in results:
    print(row_id, ":", result)
    final_matrix.append(result)

print("\\n===== FINAL MATRIX PRODUCT =====")
for row in final_matrix:
    print(row)
print("\\nMatrix multiplication completed!")
```

Key points: The manual gives only one matrix, so this example explicitly chooses A × A■. It simulates MPI Scatter-Gather with Python threads; it does not use an MPI runtime such as mpi4py. Memory: READ → SCATTER → COMPUTE → GATHER → RESULT.